



Tecnológico de Monterrey

Proyecto 4. Generador de Código

TC3002B: Desarrollo de Aplicaciones Avanzadas de Ciencias Computacionales

Grupo: 501

Alumnos:

Sylvia Fernanda Colomo Fuente | A01781983

Shaul Zayat Askenazi | A01783240

Campus:

Santa Fe

Profesores:

Victor Manuel de la Cueva Hernandez

21 de mayo de 2025

Proyecto 4. Generador de Código.....	1
Introducción.....	3
Manual de Usuario.....	4
Apéndices.....	11

Introducción

Para esta etapa del se ha optado por generar código en lenguaje ensamblador MIPS (Microprocessor without Interlocked Pipeline Stages). MIPS contiene un conjunto de instrucciones utilizado comúnmente en el ámbito de la enseñanza de arquitectura de computadoras y programación a bajo nivel. MIPS es una arquitectura RISC (Reduced Instruction Set Computer), lo que significa que emplea un conjunto reducido de instrucciones optimizadas para ejecutarse de manera eficiente, lo cual facilita su comprensión y análisis.

El funcionamiento de MIPS se basa en una estructura de registros, en la que las operaciones aritméticas, lógicas y de control se realizan directamente entre registros, en lugar de acceder constantemente a la memoria. Esto mejora el rendimiento y la eficiencia del procesamiento. MIPS utiliza un conjunto fijo de instrucciones y una arquitectura de 32 bits, donde cada instrucción ocupa exactamente 32 bits, lo que simplifica el proceso de decodificación e implementación en hardware. Además, emplea técnicas de segmentación (pipelining), permitiendo que varias instrucciones se ejecuten en paralelo en distintas etapas del procesador.

La elección de MIPS como lenguaje de ensamblador para este trabajo fue elegido por varias razones. En primer lugar, su estructura simple y regular lo convierte en una herramienta didáctica ideal para comprender conceptos fundamentales como el uso de registros, el manejo de la pila, el direccionamiento de memoria y el flujo de control. Además, existen numerosos simuladores y entornos de desarrollo, como MARS o SPIM, que permiten probar y depurar el código de forma sencilla, lo que favorece el aprendizaje práctico.

En resumen, MIPS fue seleccionado por ser una arquitectura representativa, clara y accesible, que permite concentrarse en los principios esenciales de la programación a nivel de máquina sin la complejidad añadida de otras arquitecturas más modernas.

Manual de Usuario

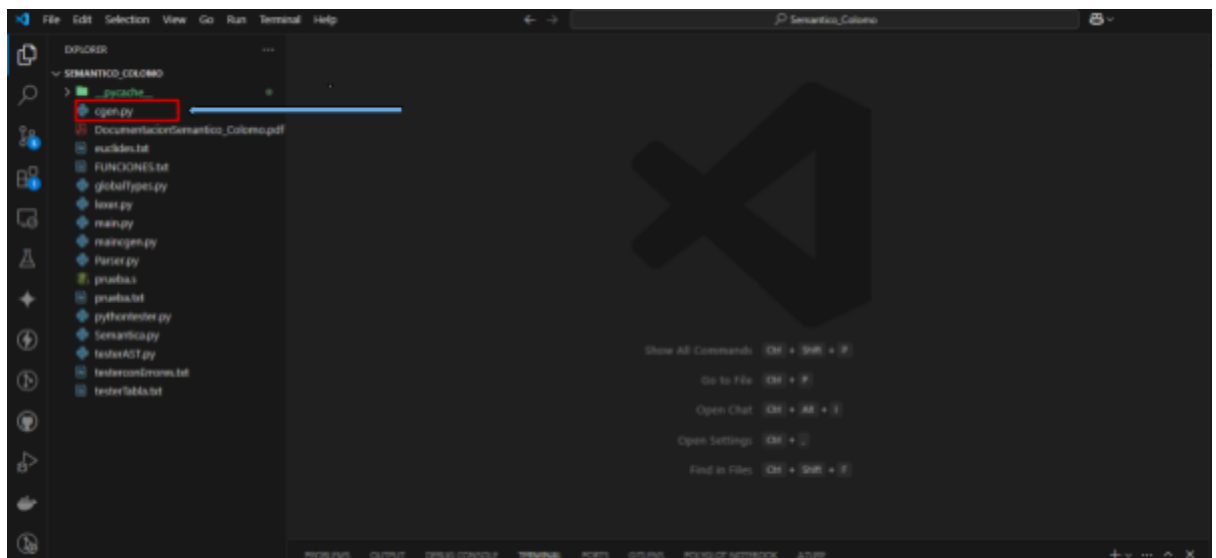
A continuación se detallan los pasos necesarios para utilizar correctamente el compilador y generar código en lenguaje ensamblador MIPS a partir de un archivo escrito en lenguaje C-:

1. Descarga y extracción del proyecto

Descargue el archivo [.zip](#) que contiene el proyecto y sus dependencias. Una vez descargado, descomprímalo en una ubicación de su preferencia.

2. Apertura del proyecto

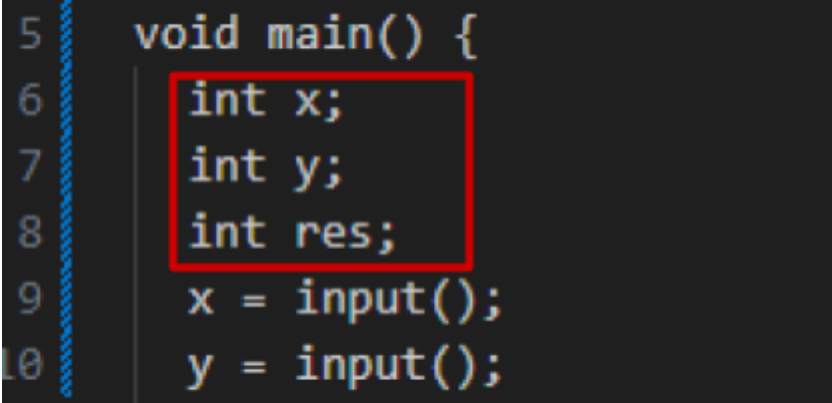
Abra el proyecto en un entorno de desarrollo o explorador de archivos, ubicándose en el directorio que contiene el archivo [cgen.py](#).



3. Preparación del archivo fuente

En ese mismo directorio, cree o copie el archivo que contenga su código fuente escrito en lenguaje C-. Guarde el archivo con extensión **.txt**. Asegúrese de que el código esté correctamente escrito y estructurado.

Nota: Todas las declaraciones de variables deben estar al inicio de cada función, antes de cualquier instrucción o sentencia de control. Esto es necesario para que el analizador semántico del compilador pueda procesar correctamente el código.



```
5 void main() {  
6     int x;  
7     int y;  
8     int res;  
9     x = input();  
10    y = input();
```

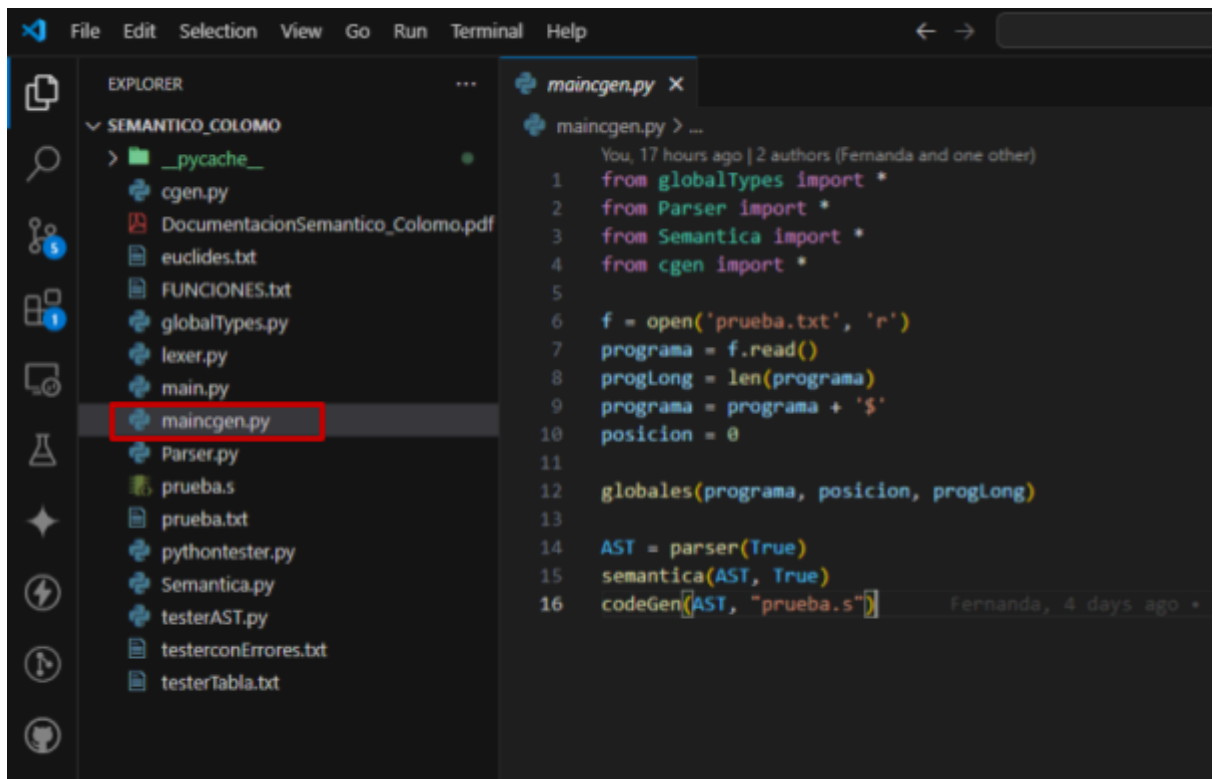
The screenshot shows a code editor with a dark background. The code is written in a light-colored font. The first line is '5 void main() {'. The next three lines are '6 int x;', '7 int y;', and '8 int res;', which are grouped together by a red rectangular box. The following two lines are '9 x = input();' and '10 y = input();'. To the left of the code, there are line numbers 5 through 10. Below the code editor, there is a file dialog box with two fields: 'Nombre de archivo:' and 'Tipo:'. The 'Nombre de archivo:' field contains the text 'prueba.txt' and the 'Tipo:' field contains the text 'Plain Text (*.txt)'.

Nombre de archivo: prueba.txt

Tipo: Plain Text (*.txt)

4. Configuración del archivo **maincgen.py**

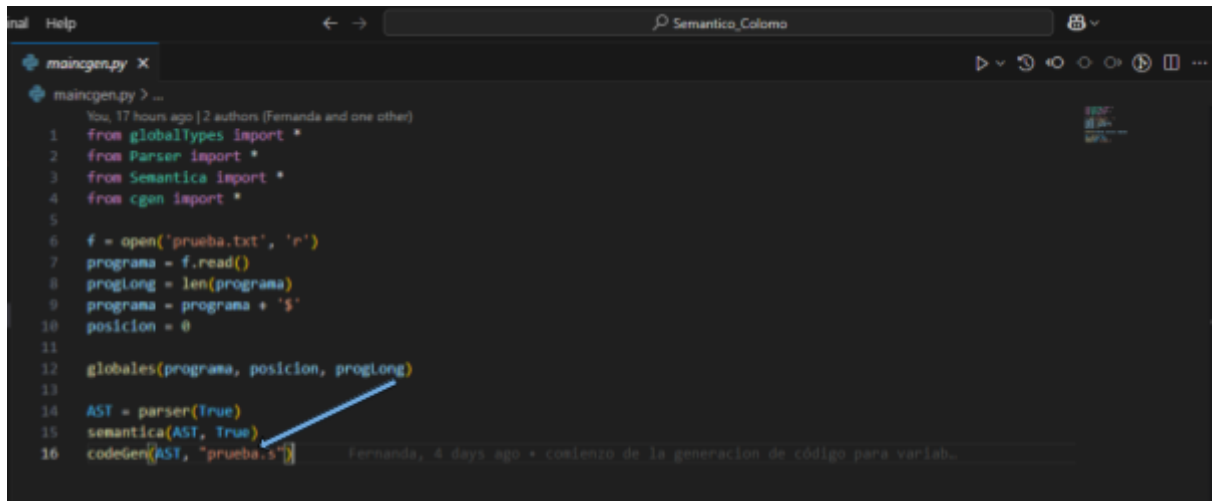
Dentro del proyecto encontrará un archivo llamado **maincgen.py**. Ábralo y edite las líneas correspondientes para:



- Especificar el **nombre del archivo de entrada** (el que contiene el código C-).



- Definir el **nombre del archivo de salida**, donde se generará el código en MIPS.



```
main Help Semantic_Coloma
maincgen.py X
maincgen.py > ...
You, 17 hours ago | 2 authors (Fernanda and one other)
1 from globaltypes import *
2 from Parser import *
3 from Semantica import *
4 from cgen import *
5
6 f = open('prueba.txt', 'r')
7 programa = f.read()
8 proglong = len(programa)
9 programa = programa + '$'
10 posicion = 0
11
12 globales(programa, posicion, proglong)
13
14 AST = parser(True)
15 semantica(AST, True)
16 codegen(AST, "prueba.s")
```

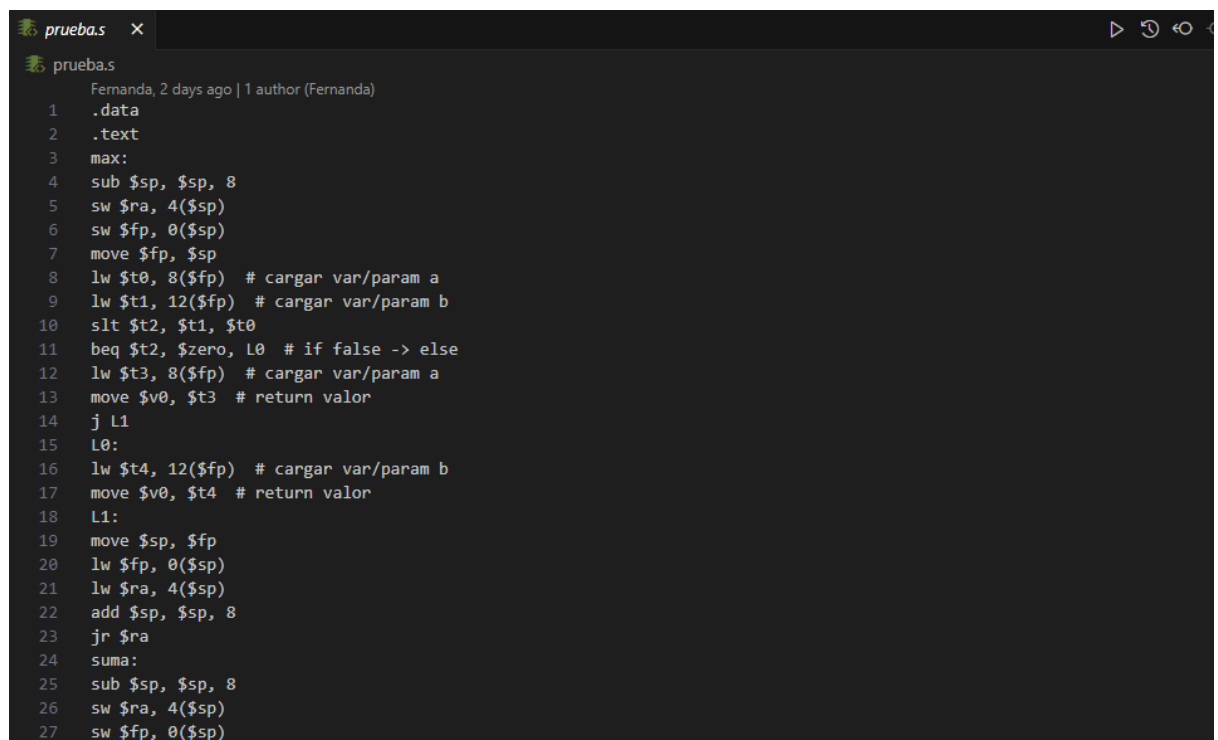
Fernanda, 4 days ago • comienzo de la generacion de código para variab...

5. Ejecución del compilador

Ejecute el archivo `maincgen.py`. Si el código fuente en C- no presenta errores léxicos, sintácticos o semánticos, se generará automáticamente el archivo de salida con el código en lenguaje ensamblador MIPS correspondiente a las instrucciones definidas en su programa.



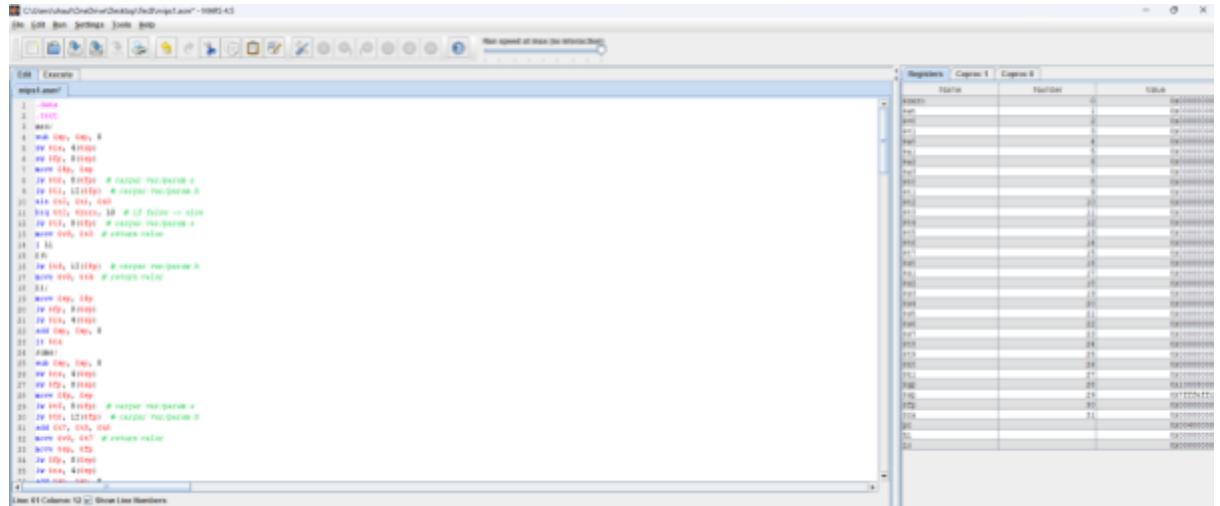
```
maincgen.py x
maincgen.py > ...
You, 17 hours ago | 2 authors (Fernanda and one other)
1 from globalTypes import *
2 from Parser import *
3 from Semantica import *
4 from cgen import *
5
6 f = open('prueba.txt', 'r')
7 programa = f.read()
8 progLong = len(programa)
9 programa = programa + '$'
10 posicion = 0
11
12 globales(programa, posicion, progLong) Fernanda, 4 days ago • comienzo de la generacion de código para variab...
13
14 AST = parser(True)
15 semantica(AST, True)
16 codeGen(AST, "prueba.s")
```



```
prueba.s x
prueba.s
Fernanda, 2 days ago | 1 author (Fernanda)
1 .data
2 .text
3 max:
4 sub $sp, $sp, 8
5 sw $ra, 4($sp)
6 sw $fp, 0($sp)
7 move $fp, $sp
8 lw $t0, 8($fp) # cargar var/param a
9 lw $t1, 12($fp) # cargar var/param b
10 slt $t2, $t1, $t0
11 beq $t2, $zero, L0 # if false -> else
12 lw $t3, 8($fp) # cargar var/param a
13 move $v0, $t3 # return valor
14 j L1
15 L0:
16 lw $t4, 12($fp) # cargar var/param b
17 move $v0, $t4 # return valor
18 L1:
19 move $sp, $fp
20 lw $fp, 0($sp)
21 lw $ra, 4($sp)
22 add $sp, $sp, 8
23 jr $ra
24 suma:
25 sub $sp, $sp, 8
26 sw $ra, 4($sp)
27 sw $fp, 0($sp)
```

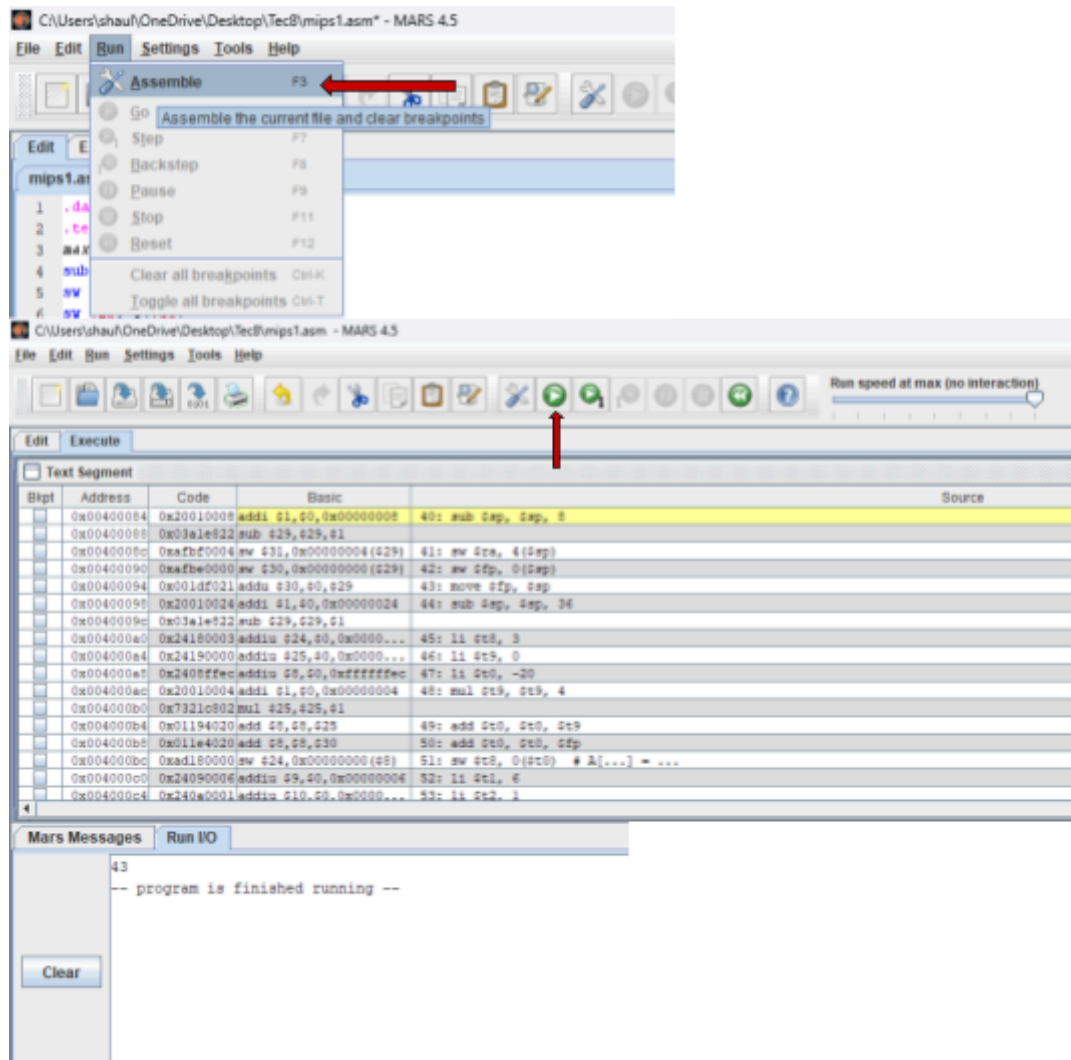

6. Carga del código en un simulador MIPS

Una vez generado el archivo MIPS, puede copiar su contenido o importarlo directamente en un simulador como **MARS** o cualquier otro compatible con la arquitectura MIPS.



7. Ensamblado y ejecución en el simulador

Cargue el archivo en MARS, ensámblelo utilizando la opción "Assemble" y, posteriormente, ejecútelo. Si el código es correcto, observará la salida que corresponde al comportamiento esperado del programa original escrito en C-.



Apéndices

[Lexer](#)

[DFA](#)

[Parser](#)

[Semántica](#)